

Documentación de Ejecución del Servidor REST

Mendoza Sánchez Javier Alberto
Aplicaciones distribuidas

1. Descripción general

El programa implementa un servidor web utilizando Node.js y el framework Express, permitiendo la creación de múltiples servicios REST para recibir y procesar solicitudes HTTP. Además, el sistema establece una conexión con una base de datos MongoDB, donde se pueden almacenar documentos mediante uno de los endpoints definidos.

El servidor expone diferentes servicios que muestran distintas formas de enviar información a una API: parámetros en la URL, parámetros en el cuerpo de la solicitud, parámetros en la ruta e inserción de documentos en una base de datos.

2. Inicialización del servidor

El servidor comienza importando las librerías necesarias y creando la aplicación Express.

```
var express = require("express");  
var app = express();  
const { MongoClient } = require("mongodb");
```

Aquí se crea el objeto 'app', el cual actúa como contenedor de todos los servicios REST que se definirán posteriormente.

Posteriormente se habilitan los middlewares para recibir datos en formato JSON.

```
app.use(express.json());  
app.use(express.urlencoded({ extended: true }));
```

3. Conexión con la base de datos

Para almacenar información, el sistema establece una conexión con MongoDB mediante el cliente oficial.

```
async function connectDB() {  
  
  const uri =
```

```
"mongodb+srv://usuario:password@cluster.mongodb.net/?appName=Cluster0";
```

```
client = new MongoClient(uri);  
  
await client.connect();  
}
```

Una vez establecida la conexión, se define la base de datos y la colección que se utilizarán.

```
function prepareDB() {  
  
  dbName = "myDatabase";  
  collectionName = "recipes";  
  
  database = client.db(dbName);  
  collection = database.collection(collectionName);  
}
```

4. Inicio del servidor

El servidor comienza a escuchar solicitudes HTTP en el puerto 3000.

```
app.listen(3000, function () {  
  
  console.log("Aplicación ejemplo, escuchando el puerto 3000!");  
  
  connectDB();  
  prepareDB();  
  
});
```

5. Endpoint raíz

Este endpoint permite verificar que el servidor se encuentra activo.

```
app.get("/", async function (request, response) {  
  
  r = {  
    message: "Nothing to send",  
  };  
});
```

```
response.json(r);  
});
```

Petición: GET http://localhost:3000

6. Servicio con parámetros en la URL

```
app.get("/serv001", async function (req, res) {
```

```
  const user_id = req.query.id;  
  const token = req.query.token;  
  const geo = req.query.geo;
```

```
  r = {  
    user_id: user_id,  
    token: token,  
    geo: geo,  
  };
```

```
  res.json(r);  
});
```

Ejemplo de solicitud: GET

http://localhost:3000/serv001?id=123&token=abc&geo=19.43,-99.13

7. Servicio con parámetros en el body (POST)

```
app.post("/serv002", async function (req, res) {
```

```
  const user_id = req.body.id;  
  const token = req.body.token;  
  const geo = req.body.geo;
```

```
  r = {  
    user_id: user_id,  
    token: token,  
    geo: geo,  
  };
```

```
  res.json(r);  
});
```

Este endpoint recibe los datos en formato JSON dentro del cuerpo de la petición.

8. Servicio con parámetros en la ruta

```
app.post("/serv003/:info", async function (req, res) {  
  
  const info = req.params.info;  
  
  let r = { info: info };  
  
  res.json(r);  
});
```

Ejemplo de solicitud: POST <http://localhost:3000/serv003/987654>

9. Inserción de documentos en MongoDB

```
app.post("/receipt/insert", async function (req, res) {  
  
  const recipes = [  
    {  
      name: "elotes cocidos",  
      ingredients: [  
        "corn",  
        "mayonnaise",  
        "cotija cheese",  
        "sour cream",  
        "lime",  
      ],  
      prepTimeInMinutes: 35,  
    },  
  ],  
  const insertManyResult = await collection.insertMany(recipes);  
  
  let r = {  
    result: insertManyResult.insertedCount  
  };  
  
  res.json(r);  
});
```

Este servicio inserta un documento en la colección 'recipes' dentro de la base de datos.

10. Conclusión

El sistema implementa un servidor REST capaz de recibir solicitudes HTTP mediante distintos métodos y formatos de entrada. Se verificó que los endpoints responden correctamente a peticiones GET y POST, permitiendo recibir parámetros desde la URL, desde el cuerpo de la solicitud o desde la ruta del endpoint. Asimismo, el servidor puede interactuar con MongoDB para almacenar documentos.